

DSA Lab Exercise: Implementation of Doubly Linked List in C

Lab Exercise

Title: Implementation of Doubly Linked List with Forward/Backward Links

Aim

To implement a **Doubly Linked List** in C and perform basic operations such as insertion, deletion, traversal, and display of the list along with the `prev`, `data`, and `next` fields of every node.

Learning Outcomes

After completing this lab, students will be able to:

1. Understand the memory representation of a doubly linked list.
 2. Create and link nodes using dynamic memory allocation.
 3. Perform insertion and deletion operations.
 4. Traverse a doubly linked list in forward and backward directions.
 5. Display the `prev`, `data`, and `next` fields of each node.
 6. Analyze the time complexity of linked-list operations.
-

Problem Statement

Write a C program to implement a **Doubly Linked List** using dynamic memory allocation.

The program should provide a menu-driven interface with the following operations:

1. Insert a node at the beginning.
2. Insert a node at the end.
3. Insert a node after a given value.
4. Delete a node from the beginning.
5. Delete a node from the end.
6. Delete a node containing a given value.
7. Display the list in forward direction.
8. Display the list in backward direction.
9. Display the complete node table showing:
 - Address of `prev`
 - `data`
 - Address of `next`
10. Exit.

Required Display

For example, if the list contains:

10 ⇌ 20 ⇌ 30 ⇌ 40

the program should display:

Forward List:

10 <-> 20 <-> 30 <-> 40 <-> NULL

and a node table similar to:

PREV	DATA	NEXT
NULL	10	0x1234
0x1000	20	0x1500
0x1200	30	0x1800
0x1500	40	NULL

Note: Actual memory addresses will be different every time the program runs.

C Program

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *head = NULL;

/* Function to create a new node */
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));
```

```

    if (newNode == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    newNode->prev = NULL;
    newNode->data = value;
    newNode->next = NULL;

    return newNode;
}

/* Insert at beginning */
void insertBeginning(int value)
{
    struct Node *newNode = createNode(value);

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }

    printf("%d inserted at beginning.\n", value);
}

/* Insert at end */
void insertEnd(int value)
{
    struct Node *newNode = createNode(value);
    struct Node *temp;

    if (head == NULL)
    {
        head = newNode;
    }
    else
    {
        temp = head;

        while (temp->next != NULL)
        {
            temp = temp->next;
        }

        temp->next = newNode;
    }
}

```

```

        newNode->prev = temp;
    }

    printf("%d inserted at end.\n", value);
}

/* Insert after a given value */
void insertAfter(int searchValue, int value)
{
    struct Node *temp = head;
    struct Node *newNode;

    while (temp != NULL && temp->data != searchValue)
    {
        temp = temp->next;
    }

    if (temp == NULL)
    {
        printf("Value %d not found.\n", searchValue);
        return;
    }

    newNode = createNode(value);

    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL)
    {
        temp->next->prev = newNode;
    }

    temp->next = newNode;

    printf("%d inserted after %d.\n", value, searchValue);
}

/* Delete from beginning */
void deleteBeginning()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;
    head = head->next;

```

```

    if (head != NULL)
    {
        head->prev = NULL;
    }

    printf("%d deleted from beginning.\n", temp->data);

    free(temp);
}

/* Delete from end */
void deleteEnd()
{
    struct Node *temp;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    if (temp->prev == NULL)
    {
        head = NULL;
    }
    else
    {
        temp->prev->next = NULL;
    }

    printf("%d deleted from end.\n", temp->data);

    free(temp);
}

/* Delete a node containing a given value */
void deleteValue(int value)
{
    struct Node *temp = head;

    while (temp != NULL && temp->data != value)
    {
        temp = temp->next;
    }
}

```

```

    if (temp == NULL)
    {
        printf("Value %d not found.\n", value);
        return;
    }

    if (temp->prev != NULL)
    {
        temp->prev->next = temp->next;
    }
    else
    {
        head = temp->next;
    }

    if (temp->next != NULL)
    {
        temp->next->prev = temp->prev;
    }

    printf("%d deleted from the list.\n", value);

    free(temp);
}

/* Display list in forward direction */
void displayForward()
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("\nForward List:\n");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->next != NULL)
            printf(" <-> ");

        temp = temp->next;
    }

    printf(" <-> NULL\n");
}

/* Display list in backward direction */

```

```

void displayBackward()
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    /* Move to last node */
    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    printf("\nBackward List:\n");

    while (temp != NULL)
    {
        printf("%d", temp->data);

        if (temp->prev != NULL)
            printf(" <-> ");

        temp = temp->prev;
    }

    printf(" <-> NULL\n");
}

/* Display node table */
void displayTable()
{
    struct Node *temp = head;

    if (head == NULL)
    {
        printf("List is empty.\n");
        return;
    }

    printf("\nDoubly Linked List Node Table\n");

    printf("-----\n");
    printf("| %-18s | %-10s | %-18s |\n",
           "PREV ADDRESS", "DATA", "NEXT ADDRESS");
    printf("-----\n");

    while (temp != NULL)
    {
        printf("| %18p | %10d | %18p |\n",

```

```

        (void*)temp->prev,
        temp->data,
        (void*)temp->next);

    temp = temp->next;
}

printf("-----\n"
}

/* Free the complete list */
void freeList()
{
    struct Node *temp;

    while (head != NULL)
    {
        temp = head;
        head = head->next;
        free(temp);
    }
}

/* Main function */
int main()
{
    int choice;
    int value;
    int searchValue;

    do
    {
        printf("\n===== \n");
        printf("      DOUBLY LINKED LIST MENU\n");
        printf("===== \n");

        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert After a Value\n");
        printf("4. Delete from Beginning\n");
        printf("5. Delete from End\n");
        printf("6. Delete a Given Value\n");
        printf("7. Display Forward\n");
        printf("8. Display Backward\n");
        printf("9. Display Node Table\n");
        printf("10. Exit\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {

```

```
case 1:
    printf("Enter value: ");
    scanf("%d", &value);
    insertBeginning(value);
    break;

case 2:
    printf("Enter value: ");
    scanf("%d", &value);
    insertEnd(value);
    break;

case 3:
    printf("Enter the value after which to insert: ");
    scanf("%d", &searchValue);

    printf("Enter new value: ");
    scanf("%d", &value);

    insertAfter(searchValue, value);
    break;

case 4:
    deleteBeginning();
    break;

case 5:
    deleteEnd();
    break;

case 6:
    printf("Enter value to delete: ");
    scanf("%d", &value);
    deleteValue(value);
    break;

case 7:
    displayForward();
    break;

case 8:
    displayBackward();
    break;

case 9:
    displayTable();
    break;

case 10:
    freeList();
    printf("Program terminated.\n");
    break;
```

```

        default:
            printf("Invalid choice!\n");
        }

    } while (choice != 10);

```

Explanation

1. Structure of a Node

The most important part of the program is:

```

C

struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};

```

Each node contains **three fields**:

```

+-----+-----+-----+
|  prev  |  data  |  next  |
+-----+-----+-----+

```

- `prev` → stores the address of the previous node.
- `data` → stores the actual value.
- `next` → stores the address of the next node.

Therefore, a doubly linked list can be represented as:

```

NULL
  ↑
prev
+-----+-----+-----+
| prev | 10 | <--> | prev | 20 | <--> | prev | 30 |
|      |   |      |      |   |      |      |
|      | next |    |      | next |    |      | next |
+-----+-----+-----+
                               ↓
                           NULL

```

More simply:

```

NULL <- [10] <-> [20] <-> [30] -> NULL

```

2. Creating a Node

The `createNode()` function dynamically allocates memory:

C

```
newNode = (struct Node*)malloc(sizeof(struct Node));
```

Then:

C

```
newNode->prev = NULL;  
newNode->data = value;  
newNode->next = NULL;
```

Initially, the new node does not point to any other node.

3. Insertion at Beginning

Suppose the list is:

```
NULL <- 20 <-> 30 -> NULL
```

Insert `10`.

The following links are changed:

C

```
newNode->next = head;  
head->prev = newNode;  
head = newNode;
```

Result:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Time Complexity

$O(1)$

4. Insertion at End

Suppose:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

To insert 40 , the program moves to the last node:

C

```
while (temp->next != NULL)
{
    temp = temp->next;
}
```

Then:

C

```
temp->next = newNode;
newNode->prev = temp;
```

Result:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

Time Complexity

O(n) because we search for the last node.

5. Insertion After a Node

Suppose:

```
10 <-> 20 <-> 40
```

We want to insert 30 after 20 .

Before:

```
20 <-----> 40
```

After:

```
20 <-----> 30 <-----> 40
```

The important statements are:

C

```
newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL)
{
    temp->next->prev = newNode;
}

temp->next = newNode;
```

Notice that **four links may need to be adjusted**.

6. Deletion from Beginning

Suppose:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

After deleting 10 :

```
NULL <- 20 <-> 30 -> NULL
```

The important statements are:

C

```
head = head->next;

if (head != NULL)
{
    head->prev = NULL;
}
```

Finally:

C

```
free(temp);
```

is used to release the memory occupied by the deleted node.

Time Complexity

$O(1)$

7. Deletion from End

The program first moves to the last node:

C

```
while (temp->next != NULL)
{
    temp = temp->next;
}
```

Then:

C

```
temp->prev->next = NULL;
```

For example:

```
10 <-> 20 <-> 30
          ↑
        delete
```

After deletion:

```
10 <-> 20 -> NULL
```

Time Complexity

$O(n)$

8. Deleting a Given Value

Suppose:

```
10 <-> 20 <-> 30 <-> 40
```

Delete 30 .

The links need to be changed from:

20 <-> 30 <-> 40

to:

20 <-----> 40

The program performs:

C

```
temp->prev->next = temp->next;
```

and:

C

```
temp->next->prev = temp->prev;
```

Then:

C

```
free(temp);
```

Time Complexity

$O(n)$

9. Forward Traversal

Forward traversal starts at `head`:

C

```
temp = head;
```

and follows:

C

```
temp = temp->next;
```

Example:

```
10 -> 20 -> 30 -> 40 -> NULL
```

Output:

Forward List:

```
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

10. Backward Traversal

For backward traversal, first move to the last node:

C

```
while (temp->next != NULL)
{
    temp = temp->next;
}
```

Then follow `prev`:

C

```
temp = temp->prev;
```

Example:

```
NULL <- 10 <-> 20 <-> 30 <-> 40
```

Output:

Backward List:

```
40 <-> 30 <-> 20 <-> 10 <-> NULL
```

This is one of the major advantages of a **doubly linked list** over a singly linked list.

11. Node Table Display

The most important part for this lab is:

C

```
printf("| %18p | %10d | %18p |\n",
      (void*)temp->prev,
      temp->data,
      (void*)temp->next);
```

The `%p` format specifier is used to display a **memory address**.

For example, the output might look like:

Doubly Linked List Node Table

PREV ADDRESS	DATA	NEXT ADDRESS
(nil)	10	0x55a8c1f012c0
0x55a8c1f012a0	20	0x55a8c1f012e0
0x55a8c1f012c0	30	0x55a8c1f01300
0x55a8c1f012e0	40	(nil)

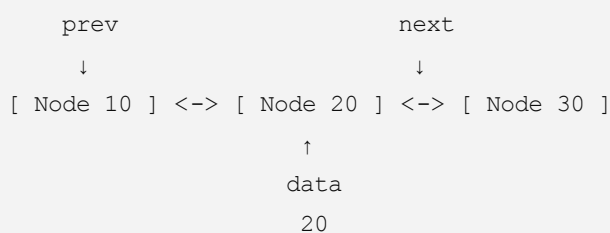
The actual addresses will vary because `malloc()` dynamically allocates memory.

How to interpret the table

For node 20 :

PREV ADDRESS	DATA	NEXT ADDRESS
0x...12a0	20	0x...12e0

This means:



Sample Execution

Suppose the student performs:

```
1 → 10
2 → 20
2 → 30
2 → 40
```

The list becomes:

```
NULL <- 10 <-> 20 <-> 30 <-> 40 -> NULL
```

Forward Display

```
Forward List:
10 <-> 20 <-> 30 <-> 40 <-> NULL
```

Backward Display

```
Backward List:
40 <-> 30 <-> 20 <-> 10 <-> NULL
```

Node Table

Doubly Linked List Node Table

	PREV ADDRESS		DATA	

	(nil)		10	
	0x12345000		20	
	0x12345678		30	
	0x12345900		40	

Time Complexity

Operation	Time Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Insert after a value	O(n)
Delete from beginning	O(1)
Delete from end	O(n)
Delete a given value	O(n)
Forward traversal	O(n)

Operation	Time Complexity
Backward traversal	$O(n)$
Display node table	$O(n)$
Search	$O(n)$

Space Complexity

For `n` nodes:

$O(n)$

because every node requires memory for:

```
prev + data + next
```

Student Tasks / Lab Questions

After implementing the above program, students should also perform these exercises:

1. Create the list:

```
10 <-> 20 <-> 30 <-> 40 <-> 50
```

and draw the complete memory representation.

2. Display the list in both forward and backward directions.
3. Insert `25` after `20` and display the node table.
4. Delete `10` and display the node table again.
5. Delete `50` and display the node table again.
6. Delete `30` and verify that both `prev` and `next` links are correctly updated.
7. Identify the address stored in the `next` field of node `20`.
8. Identify the address stored in the `prev` field of node `40`.
9. Explain why the `prev` field of the first node is `NULL`.
10. Explain why the `next` field of the last node is `NULL`.

Viva Questions

1. What is a doubly linked list?
2. What are the fields present in a doubly linked-list node?
3. What is the purpose of `prev`?

4. What is the purpose of `next` ?
5. What is the difference between singly and doubly linked lists?
6. Why is `malloc()` used?
7. Why is `free()` required?
8. What does `NULL` represent in the first node's `prev` field?
9. What does `NULL` represent in the last node's `next` field?
10. What is the advantage of backward traversal?
11. What is the time complexity of insertion at the beginning?
12. What is the time complexity of searching for a node?
13. What happens if `head == NULL` ?
14. Why must both `prev` and `next` links be updated during insertion?
15. What can happen if a link is not updated correctly?

Files, images, and data analysis are unavailable until usage resets at 1:58 PM. Continue chatting with text only, or upgrade for more access.

[Claim offer](#)